



Análise Assintótica e Força Bruta: Buscas e Ordenação por Inserção

Disciplina: Análise e complexidade de algoritmos

Caio Corrêa Santos

BI3026167

aluno em Engenharia da Computação

felix.manoel@aluno.ifsp.edu.br

Gabriel Cintra Rodrigues Montresol

BI3021955

aluno em Engenharia da Computação

felix.manoel@aluno.ifsp.edu.br

Manoel Elias Lopes Felix

BI3024971

aluno em Engenharia da Computação

felix.manoel@aluno.ifsp.edu.br

Pedro Cabelo Ferreira

BI3024342

aluno em Engenharia da Computação

pedro.cabelo@aluno.ifsp.edu.br

Murilo Vargas da Silva

Professor – Instituto Federal de Educação, Ciência e Tecnologia de São Paulo

murilo.varges@ifsp.edu.br

INTRODUÇÃO

Este relatório técnico documenta o desenvolvimento da avaliação prática realizada na disciplina de Análise e Complexidade de Algoritmos, do curso de Engenharia de Computação do IFSP Campus Birigui. O objetivo central das atividades propostas para o Grupo 1 consiste em explorar, sob perspectivas teórica e prática, os conceitos fundamentais de análise assintótica, algoritmos de força bruta, corretude e invariantes de laço.

Para tanto, o trabalho estruturou-se em torno da implementação e da análise de quatro algoritmos clássicos da literatura computacional: três métodos de busca (Busca Linear, Busca Linear em Dados Ordenados e Busca Binária) e um método de ordenação por inserção (Insertion Sort). Além do desenvolvimento dos códigos-fontes, buscou-se aplicar o rigor matemático por trás dessas estruturas, realizando a prova formal de corretude da Busca Binária via Invariante de Laço e demonstrando, a partir da definição formal da notação O , que o Insertion Sort pertence à classe de complexidade quadrática ($O(n^2)$) em seu pior caso.

Por fim, a fim de consolidar e confrontar a teoria com o comportamento real de execução, conduziu-se um experimento empírico comparativo para avaliar o desempenho temporal das buscas frente a vetores de grande escala. Complementarmente, apresenta-se a resolução analítica e a justificativa de eficiência de um desafio de programação competitiva focado em operações de busca e contagem sobre vetores ordenados.

ANÁLISE ASSINTÓTICA: NOTAÇÕES O , Ω E Θ

A análise assintótica estuda o comportamento do custo de um algoritmo quando o tamanho da entrada n tende ao infinito. Para classificar e comparar a eficiência das funções de tempo de execução, a ciência da computação utiliza as notações matemáticas Big-O (O), Big-Omega (Ω) e Big-Theta (Θ). Essas notações permitem abstrair detalhes de hardware e focar puramente na taxa de crescimento do algoritmo em larga escala.

Notação O (Big-O) — Limite Superior

A notação O é utilizada para definir o limite superior (upper bound) da taxa de crescimento de uma função. Ela estabelece o "pior cenário" ou o teto máximo de tempo que um algoritmo pode levar para executar.

Definição Teórica: Uma função $T(n)$ pertence à classe $O(g(n))$ se o tempo de execução do algoritmo não crescer mais rapidamente do que $g(n)$ multiplicado por uma constante, para valores grandes de n .

Exemplo: O pior caso do algoritmo Insertion Sort pertence à classe $O(n^2)$. Isso garante que, sob nenhuma circunstância de entrada reversa, o algoritmo terá um crescimento superior ao comportamento quadrático.

Notação Ω (Big-Omega) — Limite Inferior

A notação Ω define o limite inferior (lower bound) da taxa de crescimento de uma função. Ela descreve o "melhor cenário" ou o piso mínimo de esforço computacional que um algoritmo precisa realizar.

Definição: Uma função $T(n)$ pertence à classe $\Omega(g(n))$ se o tempo de execução do algoritmo crescer pelo menos tão rapidamente quanto $g(n)$ multiplicado por uma constante, para valores grandes de n .

Exemplo: O Insertion Sort no seu pior caso também pode ser limitado inferiormente por $\Omega(n^2)$. Isso significa que, quando o vetor está em ordem reversa, o algoritmo fará pelo menos uma quantidade quadrática de operações básicas de comparação.

Notação Θ (Big-Theta) — Limite Assintoticamente Restrito

A notação Θ é usada quando uma função é limitada tanto por cima quanto por baixo pela mesma classe de função. Ela define um limite assintoticamente restrito (tight bound), combinando as propriedades das notações O e Ω .

Definição Teórica: Dizemos que $T(n)$ pertence à classe $\Theta(g(n))$ se ela for simultaneamente $O(g(n))$ e $\Omega(g(n))$. O comportamento do algoritmo fica "espremido" dentro da mesma ordem de grandeza.

Exemplo: A Busca Binária possui complexidade de caso médio e de pior caso na classe $\Theta(\log n)$. Independentemente de o elemento estar nas extremidades ou ausente, o número de divisões sucessivas do vetor obedecerá rigidamente à curva logarítmica.

ALGORITMOS DE BUSCA: TEORIA, FUNCIONAMENTO

O problema da busca é um dos pilares fundamentais da ciência da computação e consiste em determinar se um elemento chave X pertence a um vetor A de tamanho n , retornando o seu respectivo índice de armazenamento caso seja encontrado, ou um valor sentinela (como -1) caso contrário.

BUSCA LINEAR (FORÇA BRUTA)

A Busca Linear é a abordagem mais direta e intuitiva para resolver o problema da busca. Por ser um algoritmo de força bruta, ela não impõe nenhuma pré-condição sobre a estrutura dos dados, funcionando perfeitamente em vetores completamente desordenados.

Funcionamento: O algoritmo realiza uma varredura sequencial e exaustiva. Ele inicia a verificação a partir do primeiro elemento do vetor ($A[0]$) e avança, posição por posição, comparando o elemento atual com a chave X . Esse processo continua de forma linear até que o elemento seja localizado ou que o final do vetor ($A[n-1]$) seja atingido sem sucesso.

Melhor Caso ($\Theta(1)$): Ocorre quando a chave X coincide exatamente com o primeiro elemento examinado ($A[0]$), exigindo uma única comparação.

Pior Caso ($\Theta(n)$): Acontece quando o elemento buscado está na última posição do vetor ($A[n-1]$) ou quando ele não está presente no conjunto, forçando o algoritmo a inspecionar todos os n elementos.

Caso Médio ($\Theta(n)$): Sob a premissa de uma distribuição uniforme de probabilidade (onde o elemento tem a mesma chance de estar em qualquer posição), o algoritmo precisará inspecionar, em média, metade do vetor ($\approx n/2$ elementos), mantendo uma taxa de crescimento linear.

BUSCA LINEAR EM DADOS ORDENADOS

Quando há a garantia de que o vetor de entrada está previamente ordenado (por exemplo, em ordem crescente), a busca linear clássica pode ser otimizada para diminuir o esforço computacional em cenários de ausência do elemento.

Funcionamento: O algoritmo avança de maneira sequencial idêntica à busca linear padrão. No entanto, ele ganha uma condição de parada prematura: se em alguma iteração o elemento atual do vetor for estritamente maior que a chave buscada ($A[i] > X$), o algoritmo interrompe o fluxo imediatamente e conclui que o elemento não existe. Isso é logicamente garantido porque, dado que o vetor está crescendo, nenhum elemento subsequente poderá ser igual ou menor a X .

Melhor Caso ($\Theta(1)$): Ocorre quando a chave está na primeira posição ou quando ela é estritamente menor do que o primeiríssimo elemento do vetor, disparando o critério de parada imediata.

Pior Caso ($\Theta(n)$): Acontece se a chave buscada for maior ou igual ao último elemento do vetor ($A[n-1]$), o que obriga o algoritmo a percorrer toda a estrutura até o fim.

Caso Médio ($\Theta(n)$): Embora a otimização consiga poupar metade das comparações em casos onde o elemento está ausente (interrompendo a busca no meio do caminho), o comportamento assintótico e a taxa de crescimento da função de custo em relação ao tamanho n do vetor continuam rigidamente lineares.

BUSCA BINÁRIA

A Busca Binária representa um salto qualitativo em eficiência ao aplicar o paradigma de Divisão e Conquista, sendo projetada exclusivamente para operar em vetores estritamente ordenados.

Funcionamento: Em vez de realizar uma varredura exaustiva elemento por elemento, a busca binária adota uma estratégia de eliminação em massa. O algoritmo avalia diretamente o elemento central do espaço de busca atual. Se houver equivalência com a chave X , a busca encerra-se com sucesso. Caso contrário, a ordenação dos dados permite tomar uma decisão crucial: se a chave X for menor que o elemento central, o algoritmo descarta instantaneamente toda a metade direita do vetor; se for maior, descarta a metade esquerda. O

espaço de busca é reduzido em exatamente 50% a cada iteração, e o processo se repete sucessivamente sobre o subvetor restante.

Melhor Caso ($\Theta(1)$): Ocorre na primeira tentativa, quando o elemento procurado está posicionado perfeitamente no ponto médio inicial do corte do vetor.

Pior Caso ($\Theta(\log n)$): Acontece quando o elemento está nas extremidades finais das divisões ou está ausente. Como o espaço de busca é dividido pela metade a cada passo, o número máximo de divisões até que o subvetor seja reduzido a um único elemento é ditado pela função logarítmica ($\lceil \log 2n \rceil + 1$), o que representa uma eficiência extraordinária para grandes volumes de dados.

Caso Médio ($\Theta(\log n)$): Devido à natureza da estrutura de divisões sucessivas (que pode ser visualizada como uma árvore de decisão binária), a imensa maioria dos elementos exige um número de verificações muito próximo do limite máximo, consolidando o custo médio também na classe logarítmica.

ALGORITMO DE ORDENAÇÃO POR INSERÇÃO (INSERTION SORT)

O algoritmo Insertion Sort (ou Ordenação por Inserção) é um método de ordenação clássico que constrói o arranjo final ordenado de maneira incremental. Ele é classificado como um algoritmo estável (mantém a ordem relativa de elementos com chaves iguais) e in-place (opera diretamente no vetor original, exigindo apenas $O(1)$ de memória auxiliar).

Uma analogia frequente na literatura computacional compara o funcionamento do Insertion Sort à maneira como um jogador de cartas organiza seu baralho na mão : pega-se uma carta de cada vez e, comparando-a com as que já estão organizadas, insere-se a nova carta na sua posição correta.

FUNCIONAMENTO DO ALGORITMO

O algoritmo divide virtualmente o arranjo de entrada em duas regiões distintas:

Sub Lista Ordenada: Localizada à esquerda, contendo os elementos já processados.

Sub Lista Desordenada: Localizada à direita, contendo os elementos que ainda precisam ser inseridos na região correta.

A cada iteração do laço principal, o algoritmo realiza os seguintes passos:

Seleção da Chave: O primeiro elemento da sub lista desordenada é selecionado para ser a "chave" (key) atual.

Comparação Sequencial: A chave é comparada sequencialmente, da direita para a esquerda, com os elementos que já estão na região ordenada.

Deslocamento: Enquanto os elementos anteriores forem estritamente maiores que a chave, eles são deslocados uma posição para a direita, abrindo espaço para a inserção.

Inserção Definitiva: Assim que um elemento menor ou igual à chave é encontrado (ou o início do vetor é atingido), a chave é inserida na posição que ficou vaga.

ANÁLISE DE COMPLEXIDADE ASSINTÓTICA

O comportamento temporal do Insertion Sort varia drasticamente dependendo do estado inicial de ordenação do vetor de entrada:

Melhor Caso ($\Theta(n)$): Ocorre quando o vetor já está completamente ordenado de forma crescente. Nesse cenário, o laço interno de comparação testa a chave contra o elemento imediatamente à sua esquerda e falha de primeira. Nenhum elemento precisa ser deslocado. O algoritmo realiza apenas $n-1$ comparações, resultando em uma taxa de crescimento estritamente linear.

Pior Caso ($\Theta(n^2)$): Acontece quando o vetor inicial está em ordem perfeitamente reversa (decrescente). Cada nova chave precisa ser comparada e empurrada de volta até o primeiríssimo índice do vetor, forçando o número máximo de deslocamentos. O custo total é determinado pelo somatório das comparações internas, resultando em uma complexidade quadrática.

Caso Médio ($\Theta(n^2)$): Sob a premissa de que os elementos estão distribuídos de forma aleatória, a chave precisará viajar, em média, até a metade da sublista ordenada à sua esquerda em cada passo. Matematicamente, o somatório do custo médio resulta em $4n^2-4n$, o que mantém o algoritmo na classe de complexidade quadrática.

DESENVOLVIMENTO

PROVA DE CORRETUDE DA BUSCA BINÁRIA (INVARIANTE DE LAÇO)

A prova de corretude matemática de algoritmos iterativos cujos fluxos são estruturados por laços de repetição (como o laço while da Busca Binária) exige a modelagem e a demonstração de uma Invariante de Laço. Uma invariante é uma afirmação formal sobre o estado das variáveis do algoritmo que deve permanecer obrigatoriamente verdadeira antes e depois de cada iteração.

Para a prova da Busca Binária, define-se a seguinte invariante:

Invariante de Laço: "Se o valor procurado x pertence ao vetor original $A[0 \dots n-1]$, então x obrigatoriamente está contido no intervalo fechado delimitado pelos índices do subarranjo atual $A[\text{inicio} \dots \text{fim}]$."

Segundo a metodologia formal, para comprovar a validade da invariante e atestar a corretude do algoritmo, deve-se demonstrar três propriedades fundamentais: **Inicialização**, **Manutenção** e **Término**.

INICIALIZAÇÃO

Antes da execução da primeiríssima iteração do laço, as variáveis de controle de escopo são definidas como $\text{inicio} = 0$ e $\text{fim} = n - 1$. Nesse ponto de partida, o subarranjo mapeia a totalidade do vetor original $A[0 \dots n-1]$.

Logo, se o elemento x de fato pertence ao vetor, ele está contido de maneira trivial no intervalo delimitado por esses índices. Portanto, a invariante é estritamente verdadeira no momento da inicialização.

Manutenção

Suponha que a invariante seja verdadeira imediatamente antes do início de uma iteração genérica qualquer do loop. O algoritmo calcula o índice do elemento intermediário através de $\text{meio} = \text{inicio} + (\text{fim} - \text{inicio}) / 2$ e avalia as seguintes condições de fluxo:

Caso $A[\text{meio}] == x$: O elemento alvo foi localizado. O algoritmo retorna o índice meio , o programa é encerrado imediatamente e a resposta final é exata e correta.

Caso $A[\text{meio}] < x$: Dado que o vetor A está estritamente ordenado de forma crescente, todos os elementos contidos no intervalo que vai de inicio até meio são garantidamente menores que x ($A[\text{inicio} \dots \text{meio}] < x$). Desse modo, x não pode habitar essa metade inferior. Ao atualizar o ponteiro para $\text{inicio} = \text{meio} + 1$, o novo subarranjo $A[\text{inicio} \dots \text{fim}]$ descarta a região inviável e mantém apenas os índices dos candidatos viáveis. A invariante, portanto, permanece verdadeira antes da próxima iteração.

Caso $A[\text{meio}] > x$: Pela mesma lógica da ordenação crescente, todos os elementos situados no subintervalo que vai de meio até fim são estritamente maiores do que o valor x ($A[\text{meio} \dots \text{fim}] > x$). Assim, atualiza-se o limite superior fazendo $\text{fim} = \text{meio} - 1$. O novo intervalo reduzido continua retendo o elemento x , caso ele de fato exista no vetor original. A invariante se mantém preservada.

Como o intervalo é estreitado de forma segura e sem perder o elemento de vista em nenhuma das ramificações, a propriedade de manutenção é plenamente atendida.

TÉRMINO

O laço de repetição se encerra quando a condição de controle $\text{inicio} \leq \text{fim}$ torna-se falsa, o que significa que o algoritmo atingiu um estado onde $\text{inicio} > \text{fim}$. Esse cenário resulta em um intervalo logicamente vazio (onde $\text{inicio} = \text{fim} + 1$).

Pela propriedade de preservação da invariante de laço, se o valor x estivesse presente no vetor original, ele obrigatoriamente deveria pertencer a este subarranjo corrente $A[\text{inicio} \dots \text{fim}]$. Contudo, visto que este subarranjo agora se encontra completamente vazio, conclui-se com certeza e rigor matemático que x não pertence ao vetor A .

O algoritmo encerra o fluxo retornando o valor sentinela -1 , o que é a resposta correta para o cenário de ausência do elemento. Fica demonstrada, portanto, a corretude da Busca Binária.

PROVA DE COMPLEXIDADE DO PIOR CASO DO INSERTION SORT

No capítulo de análise de algoritmos do CLRS, demonstra-se que o pior caso do *Insertion Sort* ocorre quando o vetor de entrada está em ordem reversa. A função que descreve o custo de tempo do pior caso pode ser expressa pelo somatório das comparações feitas pelo laço interno:

$$T(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

A definição formal afirma que $T(n) = \Theta(n^2)$ se existirem constantes reais positivas c_1, c_2 e um inteiro $n \geq 0$ tais que:

$$0 \leq c_1 n^2 \leq T(n) \leq c_2 n^2$$

LIMITE SUPERIOR — NOTAÇÃO $O(n^2)$

Precisamos encontrar um c_2 tal que $\frac{n^2}{2} - \frac{n}{2} \leq c_2 n^2$. Como o termo $-\frac{n}{2}$ é sempre negativo ou zero para $n \geq 1$, podemos afirmar logicamente que:

$$\frac{n^2}{2} - \frac{n}{2} \leq \frac{n^2}{2}$$

Portanto, ao escolhermos $c_2 = \frac{1}{2}$, a inequação é satisfeita para todo $n \geq 1$. Concluimos que $T(n) = O(n^2)$.

LIMITE INFERIOR — NOTAÇÃO $\Omega(n^2)$

Precisamos encontrar um c_1 tal que $\frac{n^2}{2} - \frac{n}{2} \geq c_1 n^2$. Dividindo todos os termos por n^2 , temos a inequação:

$$\frac{1}{2} - \frac{1}{2n} \geq c_1$$

Para valores de $n \geq 2$, o termo $\frac{1}{2n}$ será sempre menor ou igual a $\frac{1}{4}$. Substituindo esse valor máximo na equação para analisar o caso limite, obtemos:

$$\frac{1}{2} - \frac{1}{4} = \frac{1}{4}$$

Portanto, escolhendo a constante $c_1 = \frac{1}{4}$ e assumindo que $n \geq 2(n_0 = 2)$, a desigualdade é garantida. Concluimos que $T(n) = \Omega(n^2)$.

EXPERIMENTO EMPÍRICO DOS ALGORITMOS DE BUSCA (C++)

O experimento empírico teve como objetivo avaliar o tempo de execução real dos algoritmos de busca sob as piores condições possíveis. Para isso, foram utilizados vetores ordenados com diferentes ordens de grandeza:

$$n = 10^3, 10^4, 10^5 \text{ e } 10^6$$

DEFINIÇÃO DO PIOR CASO

O elemento procurado foi definido como:

$$X = n + 1$$

Como esse valor é maior que todos os elementos presentes no vetor e não pertence ao conjunto de dados, cada algoritmo é forçado a executar sua quantidade máxima de operações:

Busca Linear: percorre todos os elementos do vetor, realizando n comparações no pior caso ($O(n)$).

Busca Linear em Vetor Ordenado: também percorre todo o vetor, pois o valor procurado é maior que todos os elementos existentes ($O(n)$).

Busca Binária: realiza o número máximo de divisões sucessivas do vetor até esgotar o intervalo de busca ($O(\log n)$).

Metodologia Experimental

Para a coleta dos tempos de execução foi utilizada a biblioteca padrão <chrono> da linguagem C++.

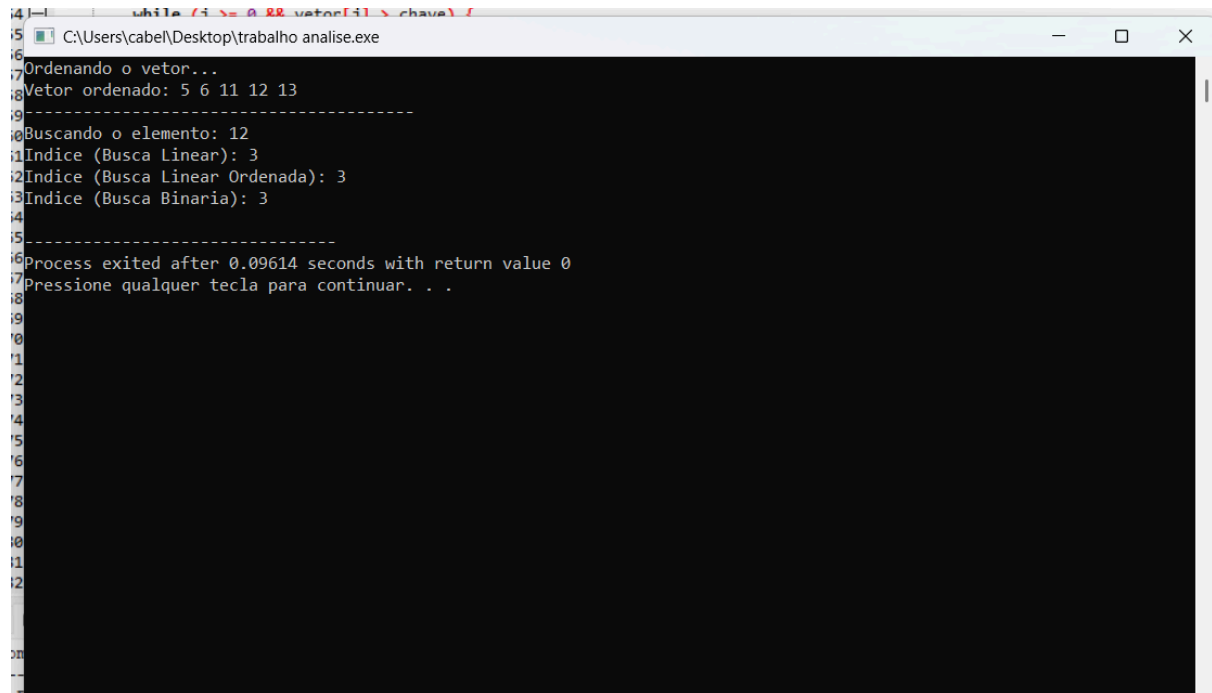
Como o tempo de execução da busca binária é extremamente pequeno para os tamanhos analisados, cada algoritmo foi executado repetidamente, e o tempo total foi medido. Essa abordagem reduz erros de medição e permite uma comparação mais precisa entre os algoritmos.

Os experimentos foram realizados utilizando vetores ordenados de tamanhos 10^3 , 10^4 , 10^5 e 10^6 , contendo valores sequenciais. O elemento buscado foi definido como $X = n + 1$, garantindo que ele não estivesse presente no vetor e forçando a execução do pior caso para cada algoritmo.

OBJETIVO DA ANÁLISE

Os resultados obtidos permitem comparar o comportamento observado experimentalmente com as previsões da análise assintótica estudada em sala de aula, evidenciando a diferença prática entre algoritmos de complexidade linear $O(n)$ e logarítmica $O(\log n)$

Abaixo está a captura de tela demonstrando o funcionamento e a validação dos algoritmos de ordenação implementados:



```
4 | while (i <= 0 && vetor[i] < chave) {
5 | }
6 |
7 | Ordenando o vetor...
8 | Vetor ordenado: 5 6 11 12 13
9 | -----
10 | Buscando o elemento: 12
11 | Indice (Busca Linear): 3
12 | Indice (Busca Linear Ordenada): 3
13 | Indice (Busca Binaria): 3
14 | -----
15 |
16 | Process exited after 0.09614 seconds with return value 0
17 | Pressione qualquer tecla para continuar. . .
18 |
19 |
20 |
21 |
22 |
23 |
24 |
25 |
26 |
27 |
28 |
29 |
30 |
```

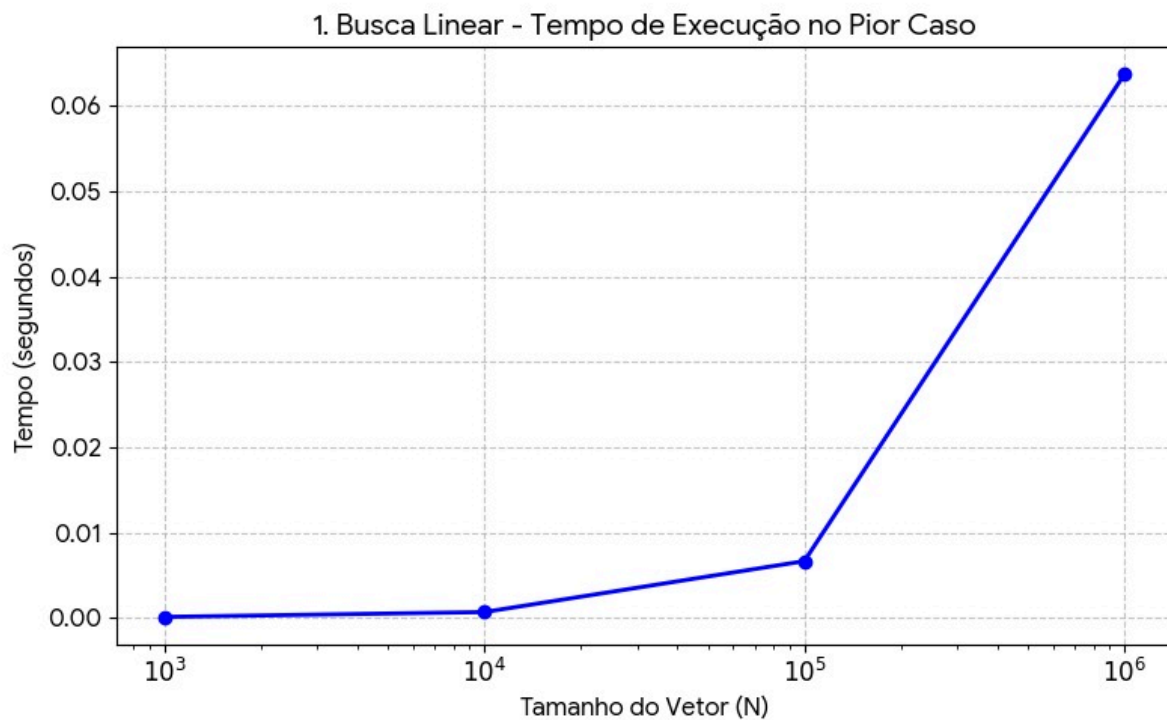
Fonte: Autoria própria

Tempo de execução(ms)			
Tamanho do Vetor	Busca Linear	Busca Linear Ordenada	Busca Binário
$10^3 = 1000$	0	0	0
$10^4 = 10000$	31,5291	31,538	0
$10^5 = 100000$	254,082	349,234	0
$10^6 = 1000000$	2486,082	3664,23	0

Fonte: Autoria própria

RESULTADOS GRÁFICOS

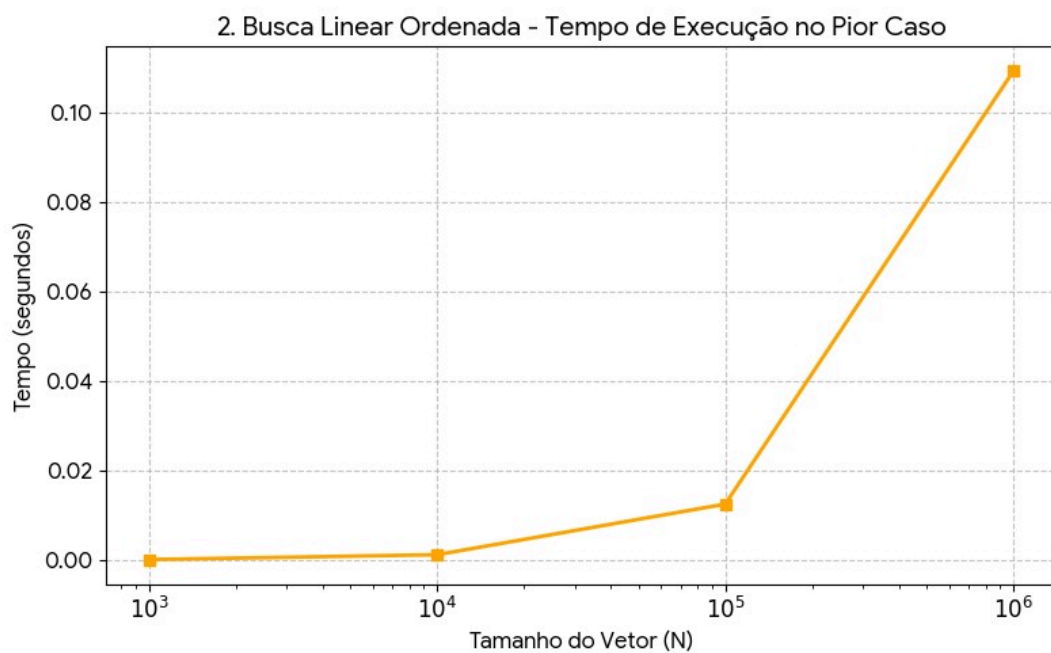
BUSCA LINEAR



Fonte: Autoria própria

Observa-se um crescimento aproximadamente linear do tempo de execução conforme o tamanho do vetor aumenta, comportamento compatível com a complexidade teórica $O(n)$.

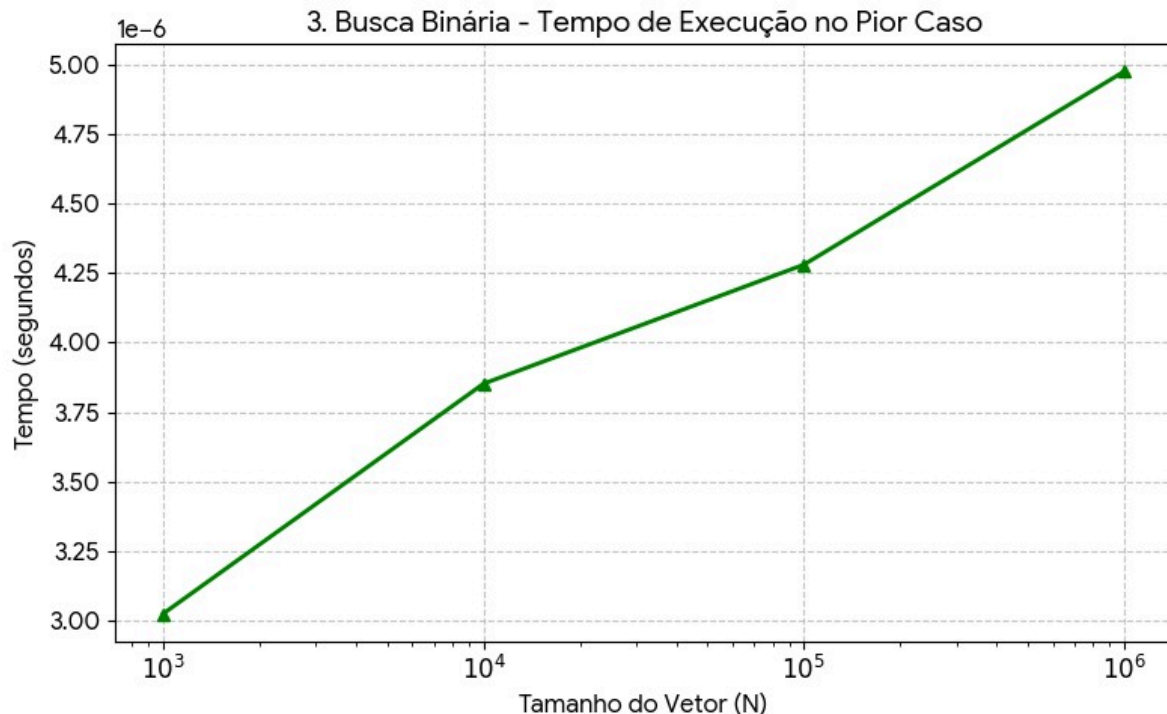
BUSCA LINEAR ORDENADA



Fonte: Autoria própria

Como o elemento procurado é maior que todos os elementos do vetor, a busca linear ordenada também percorre todas as posições, apresentando comportamento linear $O(n)$.

BUSCA BINÁRIA



Fonte: Autoria própria

Mesmo para vetores com um milhão de elementos, o tempo de execução permanece praticamente constante quando comparado às buscas lineares. Isso ocorre porque a busca binária possui complexidade $O(\log n)$, necessitando de apenas algumas divisões sucessivas do vetor para concluir que o elemento não existe.

RESOLUÇÃO DO DESAFIO DE PROGRAMAÇÃO COMPETITIVA

A análise a seguir é feita sobre a implementação abaixo, que ordena o vetor uma única vez e resolve cada consulta com `lower_bound` (busca binária da STL).

Análise de complexidade da solução

O programa executa três blocos de trabalho distintos para cada caso de teste

Etapa	Custo
Leitura dos N elementos do vetor	$O(N)$
<code>sort(marbles.begin(), marbles.end())</code>	$O(N \log N)$
Q chamadas a <code>lower_bound</code> (uma por consulta)	$O(Q \log N)$

Fonte: Autoria própria

CUSTO TOTAL POR CASO DE TESTE:

$$O(N) + O(N \log N) + O(Q \log N)$$

O termo $O(N)$ é dominado por $O(N \log N)$ (pois $\log N > 1$ para $N > 1$), logo ele desaparece na notação assintótica. A complexidade final do algoritmo é, portanto

$$O((N + Q) \log N)$$

POR QUE A BUSCA BINÁRIA (LOWER_BOUND) É $O(\log N)$

A busca binária mantém um invariante: a cada iteração, o intervalo de busca `[low, high]` é reduzido aproximadamente à metade. Seja $T(n)$ o número de comparações necessárias para um vetor de tamanho n . A recorrência que descreve o algoritmo é:

$$T(n) = T(n / 2) + O(1)$$

Essa é uma recorrência clássica de divisão e conquista, resolvida pelo Teorema Master (caso em que $a = 1$, $b = 2$ e o trabalho extra por nível é constante). A solução é

$$T(n) = O(\log_2 n)$$

Intuição da prova: a cada passo, o tamanho do espaço de busca é dividido por 2. Após k divisões, o espaço de busca tem tamanho aproximado $n / 2^k$. O algoritmo termina quando esse espaço atinge tamanho menor ou igual a 1, isto é, quando:

$$\frac{n}{2^k} \leq 1 \Rightarrow 2^k \geq n \Rightarrow k \geq \log_2 n$$

Logo, o número de iterações (e, portanto, de comparações) é proporcional a $\log_2 n$, o que

justifica formalmente a complexidade $O(\log N)$ de cada chamada a `lower_bound`

Conclusão Experimental

Os resultados obtidos confirmam empiricamente as previsões da análise assintótica:

As buscas Linear e Linear Ordenada apresentaram crescimento proporcional ao tamanho da entrada ($O(n)$).

A Busca Binária apresentou crescimento extremamente reduzido ($O(\log n)$).

À medida que o tamanho do vetor aumenta, a diferença de desempenho entre os algoritmos torna-se cada vez mais significativa.

*Houve auxílio de inteligência artificial para escrita e correção ortográfica do documento.

REFERÊNCIAS E LINKS

GIT-HUB: <https://github.com/PedroC-dev/analise-assintotica-fcbruta.git>

Cormen, T. Desmistificando algoritmos, Vol. 1. Elsevier Brasil, 2017.

Sipser, M. Introdução à teoria da computação, 2ª ed. Thomson Learning, 2007.

Vieira, L. T.; Veloso, P. A. S. Complexidade de Algoritmos, Vol. 3. Bookman, 2013.

QUEM DISSE, Carla. **Corretude de algoritmos (iterativos) - Invariante de laço**. YouTube, 7 fev. 2022. Disponível em: <https://youtu.be/AQ7A2Z0TdM4>. Acesso em: 24 jun. 2026.]

FELIPE, Henrique. **Insertion Sort**. Blog Cyberini, 25 jun. 2018. Disponível em: <https://www.blogcyberini.com/2018/06/insertion-sort.html>. Acesso em: 24 jun. 2026.